

TECH SCHOOL

The motivation of gRPC

Xin chào và chào mừng các bạn đến với Tech School.

Trong khóa học này, chúng ta sẽ tìm hiểu về gRPC và cách sử dụng nó kết hợp với Protocol Buffers để phát triển ứng dụng bằng Go và Java.

OK, hãy bắt đầu với một câu hỏi đơn giản:

Động lực ra đời của gRPC là gì?

Hay nói cách khác, gRPC được tạo ra để giải quyết vấn đề gì?

Câu trả lời là: giao tiếp (communication).

Ngày nay, các ứng dụng được viết bằng rất nhiều ngôn ngữ lập trình khác nhau.

Ví dụ:

- Backend có thể được viết bằng Go
- Ứng dụng Android được viết bằng Java
- Ứng dụng iOS được viết bằng Swift

Vậy làm thế nào để chúng giao tiếp với nhau?

Xu hướng hiện nay là sử dụng kiến trúc microservices.

Ngay cả ở phía backend, chúng ta cũng có thể có rất nhiều service khác nhau, được viết bằng các ngôn ngữ khác nhau như Go, Python, Rust, tùy thuộc vào nhu cầu kinh doanh và các ràng buộc kỹ thuật.

Để các service này có thể giao tiếp với nhau, chúng bắt buộc phải thống nhất về một tập các API contract, ví dụ như:

- Kênh giao tiếp
- Cơ chế xác thực
- Định dạng dữ liệu truyền tải (payload)
- Mô hình dữ liệu
- Cách xử lý lỗi

Có rất nhiều yếu tố cần phải xem xét.

Và đó là lý do tại sao việc xây dựng một API là rất khó.

Không chỉ vậy, chúng ta còn muốn việc giao tiếp này phải hiệu quả, tức là nhanh và nhẹ.

Bạn biết đấy, số lượng message được trao đổi giữa các microservice là cực kỳ lớn, nên giao tiếp càng nhanh thì càng tốt.

Ngoài ra, trong một số môi trường như ứng dụng di động, nơi mà tốc độ mạng và băng thông bị hạn chế, việc sử dụng một giao thức giao tiếp nhẹ để tương tác với server là vô cùng quan trọng.

Cuối cùng nhưng không kém phần quan trọng, chúng ta muốn việc giao tiếp phải đơn giản.

Hãy tưởng tượng chúng ta đang xây dựng một hệ thống có hàng trăm hoặc thậm chí hàng nghìn microservice.

Chắc chắn chúng ta không muốn dành phần lớn thời gian chỉ để viết code giúp chúng nói chuyện với nhau, đúng không?

Điều chúng ta mong muốn là có một framework nào đó cho phép developer:

- Tập trung vào việc triển khai logic cốt lõi của service

- Và để framework lo toàn bộ những phần còn lại như giao tiếp, serialization, error handling, v.v.

Và framework đó chính là gRPC.

OK, đến đây bạn đã có một số ý tưởng về những vấn đề mà gRPC được tạo ra để giải quyết.

Trong bài giảng tiếp theo, chúng ta sẽ tìm hiểu chính xác gRPC là gì và nó hoạt động như thế nào để đạt được mục tiêu đó.

What is gRPC and how it works

Chào mừng các bạn quay trở lại!

Trong bài giảng trước, chúng ta đã biết rằng gRPC được tạo ra để giải quyết vấn đề giao tiếp.

Vì vậy bây giờ, hãy cùng tìm hiểu gRPC hoạt động như thế nào.

gRPC là gì?

gRPC là một framework mã nguồn mở, hiệu năng cao và giàu tính năng.

Ban đầu, nó được phát triển bởi Google, và hiện nay đã trở thành một phần của Cloud Native Computing Foundation (CNCF) — tương tự như Kubernetes hay Prometheus.

Vậy chữ “G” trong gRPC có nghĩa là gì?

Ban đầu, tôi từng nghĩ nó là Google. Ừ thì... bạn biết tại sao rồi đấy. Nhưng thực tế thì không phải vậy.

Trên thực tế, chữ G mang những ý nghĩa khác nhau trong từng phiên bản gRPC, chẳng hạn như: good, green, glorious, và thậm chí là gRPC itself.

Bạn có thể xem danh sách đầy đủ ở đường link được cung cấp.

Thế còn RPC thì sao?

RPC là viết tắt của Remote Procedure Call (Gọi thủ tục từ xa).

Đây là một giao thức cho phép một chương trình thực thi một hàm (procedure) của một chương trình khác, chương trình đó có thể nằm trên một máy tính khác.

Điều tuyệt vời ở RPC là:

Lập trình viên không cần phải tự code chi tiết về giao tiếp mạng.

Tất cả những phần đó được framework bên dưới tự động xử lý.

Vì vậy, trong code phía client, nó trông giống như chúng ta đang gọi trực tiếp một hàm của server.

Và nó vẫn hoạt động ngay cả khi client và server được viết bằng những ngôn ngữ khác nhau.

Ví dụ:

- Client được viết bằng Go

- Server được viết bằng Rust

gRPC làm được điều đó như thế nào?

Về cơ bản, phía client sẽ có một stub.

Stub này cung cấp các method hoặc function giống hệt như phía server.

Stub này được gRPC tự động sinh ra cho bạn.

Bên trong, stub sẽ gọi framework gRPC để:

- Đóng gói dữ liệu
- Gửi request qua mạng
- Nhận response từ server

Mọi thứ hoạt động một cách “kỳ diệu”.

Nhờ có stub, client và server chỉ cần quan tâm đến việc triển khai logic cốt lõi của service, còn tất cả những chi tiết phức tạp về giao tiếp mạng đã được gRPC lo hết.

Trong bài giảng tiếp theo, chúng ta sẽ tìm hiểu cách các gRPC stub được sinh ra, với sự trợ giúp của Protocol Buffers.

gRPC code generation with Protocol Buffer

Sinh code (Code generation) là một trong những tính năng quan trọng nhất của gRPC. Trong bài giảng này, chúng ta sẽ tìm hiểu gRPC thực hiện việc sinh code như thế nào với Protocol Buffers.

Để có thể sinh ra stub cho server và client, trước hết chúng ta cần viết API contract.

API contract này bao gồm:

- Mô tả các service
- Và các message payload

Tất cả được định nghĩa trong một file Protocol Buffer (proto) trông giống như thế này.

Trong file này, chúng ta định nghĩa một method tên là Hello. Method này:

- Nhận vào một HelloRequest
- Và trả về một HelloResponse

HelloRequest chỉ chứa một field string là name.

HelloResponse chứa một field string là greet.

Rất đơn giản, đúng không?

Chúng ta sẽ tìm hiểu chi tiết hơn rất nhiều về Protocol Buffers trong bài hands-on với Protocol Buffer sau này.

Từ file proto này, code stub cho server và client sẽ được sinh ra bởi Protocol Buffer Compiler, hay còn gọi là protoc.

Tùy vào ngôn ngữ lập trình mà chúng ta sử dụng, chúng ta cần chỉ định cho compiler biết plugin gRPC tương ứng.

Vì sao gRPC sử dụng Protocol Buffers?

Có rất nhiều lý do:

Thứ nhất, rất dễ đọc và dễ hiểu.

Thứ hai, đa ngôn ngữ (language interoperable), với khả năng tự động sinh code cho rất nhiều ngôn ngữ lập trình khác nhau.

Thứ ba, Protocol Buffer biểu diễn dữ liệu dưới dạng nhị phân (binary), nên kích thước nhỏ hơn, truyền nhanh hơn, và serialize hiệu quả hơn so với các định dạng dạng text như JSON hay XML.

Thứ tư, nó cung cấp một API contract có kiểu dữ liệu chặt chẽ (strongly typed) giữa client và server, giúp làm việc an toàn hơn rất nhiều.

Cuối cùng, Protocol Buffer có một bộ quy tắc rất tốt cho việc phát triển API, đảm bảo tương thích ngược và tương thích xuôi (backward & forward compatibility).

Nghe khá là tuyệt vời, đúng không?

Protocol Buffer có bắt buộc khi dùng gRPC không?

Câu trả lời là: Không.

Bạn hoàn toàn có thể sử dụng:

- Google FlatBuffers
- Hoặc Microsoft Bond

Tuy nhiên, theo quan điểm cá nhân của tôi, Protocol Buffer vẫn là một lựa chọn cực kỳ tốt.

Nó hỗ trợ rất nhiều ngôn ngữ lập trình, trong đó:

- 10 ngôn ngữ được hỗ trợ chính thức
 - + Go, Java, NodeJS được triển khai thuần (pure implementation)
 - + 7 ngôn ngữ còn lại là wrapper của gRPC-core viết bằng C, bao gồm: C++, C#, Objective-C, Python, Ruby, Dart và PHP

Ngoài ra, còn có rất nhiều thư viện không chính thức cho các ngôn ngữ khác như:

- Swift
- Rust
- TypeScript
- Haskell

và nhiều ngôn ngữ khác

OK, bây giờ chúng ta đã biết gRPC sinh code cho chúng ta như thế nào. Hy vọng bạn thấy phần này thú vị.

Trong bài giảng tiếp theo, chúng ta sẽ khám phá một “vũ khí bí mật” khác của gRPC, đó chính là HTTP phiên bản 2 (HTTP/2). Hẹn gặp lại các bạn ở bài sau!

HTTP-2 - The secret weapon of gRPC

Xin chào và chào mừng các bạn quay trở lại với khóa học gRPC.

Trong bài giảng này, chúng ta sẽ tìm hiểu về HTTP/2 và điều gì khiến nó trở nên tuyệt vời.

OK, cùng bắt đầu nhé!

1. gRPC và HTTP/2

gRPC sử dụng HTTP/2 làm giao thức truyền tải (Transfer Protocol). Vì vậy, nó kế thừa rất nhiều tính năng mạnh mẽ mà HTTP/2 mang lại.

Một trong số đó là binary framing (đóng gói dữ liệu dạng nhị phân), giúp hệ thống:

- Hiệu năng cao
- Ổn định
- Truyền tải nhẹ hơn
- Giải mã an toàn hơn

so với các giao thức dựa trên text.

Và bởi vì HTTP/2 là nhị phân, nó kết hợp cực kỳ tốt với Protocol Buffers.

HTTP/2 còn nén header bằng cơ chế HPACK, giúp giảm chi phí overhead và cải thiện hiệu năng.

2. Multiplexing

HTTP/2 hỗ trợ multiplexing, nghĩa là client và server có thể gửi nhiều request và response song song trên một kết nối TCP duy nhất.

Điều này giúp:

- Giảm độ trễ (latency)
- Tận dụng mạng tốt hơn

3. Server Push

Cuối cùng, HTTP/2 hỗ trợ server push. Chỉ với một request từ client, server có thể gửi về nhiều response.

Điều này cực kỳ hữu ích trong việc giảm round-trip latency giữa client và server trong nhiều trường hợp.

Đặc biệt khi server biết chính xác những tài nguyên mà client sẽ cần, và gửi chúng trước ngay cả khi client chưa yêu cầu.

4. Demo: HTTP/2 nhanh hơn HTTP/1.1 như thế nào?

Bây giờ, hãy xem một demo để so sánh tốc độ giữa HTTP/2 và HTTP/1.1.

Trong demo này, chúng ta sẽ tải 200 ảnh nhỏ từ server.

Tôi sẽ refresh trang này.

- Với HTTP/1, mất 1.26 giây
- Bây giờ thử với HTTP/2, chỉ mất 0.62 giây.

Như bạn thấy, HTTP/2 nhanh gần gấp đôi HTTP/1.

5. HTTP/2 hoạt động như thế nào bên trong?

Cấu trúc logic của HTTP/2 có thể được mô tả như trong hình minh họa.

- Có một kết nối TCP duy nhất
- Trên đó chứa nhiều stream hai chiều (bi-directional streams)
- Mỗi stream có một ID riêng
- Mỗi stream mang nhiều message hai chiều

Mỗi message (request hoặc response) được chia nhỏ thành nhiều frame nhị phân.

Frame là đơn vị nhỏ nhất, chứa các loại dữ liệu khác nhau như:

- headers
- settings
- priority
- data
- v.v.

Trên thực tế, các stream không chảy riêng biệt, mà các frame của chúng được xen kẽ (interleaved)

trên cùng một kết nối TCP, và sẽ được lắp ráp lại khi đến phía bên kia.

Chính nhờ lớp binary framing này, mà stream multiplexing mới có thể thực hiện được trong HTTP/2.

6. So sánh HTTP/2 và HTTP/1.1

Bây giờ, hãy so sánh HTTP/2 với HTTP/1.1:

HTTP/2 là giao thức nhị phân, trong khi HTTP/1.1 là text-based

Header trong HTTP/2 được nén, còn HTTP/1.1 là plain text

HTTP/2 hỗ trợ multiplexing, HTTP/1.1 thì không
HTTP/2 có thể gửi nhiều request/response trên một kết nối
HTTP/1.1 chỉ gửi được một request tại một thời điểm, nên phải mở nhiều kết nối TCP nếu muốn gửi nhiều request
HTTP/2 hỗ trợ server push, HTTP/1.1 thì không
HTTP/2 được phát hành vào năm 2015, HTTP/1.1 được phát hành từ năm 1997
OK, như vậy là chúng ta đã kết thúc bài giảng về HTTP/2.
Trong bài tiếp theo, chúng ta sẽ tìm hiểu các use case của gRPC và so sánh gRPC với REST.
Cảm ơn các bạn đã theo dõi, và hẹn gặp lại sớm!

Is gRPC better than REST

Xin chào và chào mừng các bạn quay trở lại!

Trong bài giảng này, chúng ta sẽ cùng tìm hiểu các use case của gRPC và so sánh gRPC với REST.

1. Các loại gRPC

Có 4 loại gRPC:

- Unary RPC: đây là loại đơn giản nhất. Client gửi một request duy nhất, và server trả về một response duy nhất. Cách này khá giống với API HTTP thông thường.

- Client Streaming: trong trường hợp này, client gửi một luồng gồm nhiều message, và server chỉ trả về một response duy nhất.

Server Streaming: ngược lại với client streaming, client chỉ gửi một request, còn server trả về một luồng gồm nhiều response.

Bi-directional Streaming (Bidirectional streaming): đây là loại phức tạp nhất. Client và server có thể gửi và nhận nhiều message song song, với thứ tự bất kỳ. Cách này rất linh hoạt và không bị blocking, nghĩa là không bên nào cần phải chờ phản hồi trước khi gửi message tiếp theo.

Đó là cái nhìn tổng quan ở mức cao về 4 cách giao tiếp trong gRPC.

Chúng ta sẽ quay lại phần này trong các bài hands-on, nơi chúng ta sẽ triển khai từng loại để hiểu sâu hơn.

2. So sánh gRPC và REST

Bây giờ, hãy so sánh nhanh gRPC với REST để thấy sự khác biệt.

- gRPC sử dụng HTTP/2, nhanh hơn nhiều so với HTTP/1.1 mà REST thường dùng.

- gRPC sử dụng Protocol Buffers để serialize dữ liệu, là nhị phân và kích thước nhỏ.

Trong khi đó REST dùng JSON, là dạng text và lớn hơn.

- API contract trong gRPC là chặt chẽ, và bắt buộc phải được định nghĩa rõ ràng trong file proto. Với REST, API contract thường lỏng lẻo và không bắt buộc, có thể dùng OpenAPI nếu muốn nhưng không phải yêu cầu bắt buộc.

- Code generation được tích hợp sẵn trong gRPC thông qua protocol buffer compiler.

Với REST, ta thường phải dùng công cụ bên thứ ba như OpenAPI hoặc Swagger.

- Cả gRPC và REST đều hỗ trợ bảo mật bằng SSL/TLS.

- gRPC hỗ trợ streaming hai chiều, trong khi REST chỉ hỗ trợ request một chiều từ client lên server.

Như vậy, xét trên hầu hết các khía cạnh, gRPC vượt trội hơn REST.

Tuy nhiên, vẫn có một điểm mà REST làm tốt hơn, đó là hỗ trợ trình duyệt (browser support).

REST được tất cả các trình duyệt hỗ trợ đầy đủ, trong khi gRPC chưa được hỗ trợ trực tiếp, và cần sử dụng gRPC-Web kèm theo một proxy layer để chuyển đổi giữa HTTP/1 và HTTP/2.

3. Khi nào nên dùng gRPC?

gRPC có rất nhiều điểm mạnh, nhưng nó cũng có những hạn chế riêng. Vậy khi nào và ở đâu chúng ta nên sử dụng gRPC để tận dụng tối đa lợi ích của nó?

- Microservices: đây là nơi gRPC thực sự tỏa sáng, nhờ khả năng giao tiếp độ trễ thấp, thông lượng cao, và API contract chặt chẽ.

- Môi trường đa ngôn ngữ (polyglot): gRPC rất phù hợp vì nó hỗ trợ sinh code tự động cho nhiều ngôn ngữ lập trình khác nhau.

- Giao tiếp thời gian thực, point-to-point gRPC là lựa chọn tốt nhờ hỗ trợ bi-directional streaming rất mạnh.

- Môi trường hạn chế về mạng, chẳng hạn như ứng dụng mobile, vì gRPC sử dụng định dạng message nhẹ, tiết kiệm băng thông.

Vậy là bạn đã hoàn thành toàn bộ phần lý thuyết của khóa học gRPC. Chúc mừng bạn! Hy vọng bạn thấy khóa học thú vị và hữu ích. Hẹn gặp lại các bạn trong các bài hands-on thực hành tiếp theo.